

挙動調査まとめ

対象: 受注取り込み/ライン展開/
生産計画入力/在庫・残量一覧

1. 操作別の挙動・参照・保存

1) 受注取り込み (/csv-upload)

操作	API	参照 (テーブル/列)	計算/処理	保存先 (テーブル/列)
CSVアップロード	POST /stg-order-raw/ upload_csv/ または upload_hirakat a_special	CSV行: 取込サービスに依 存 (汎用: product_code, d ue_date, quantity な ど)	文字コード判定 → Raw保存 → Daily展開 → 受注作成	StgOrderRaw* (得意先 により StgOrderRaw/Kubota/T iera/Rieden を使用) StgOrderDaily.custom er/order_type/produc t_code/due_date/quan tity/plant_code/ship _to_code/source_file Order.* OrderLine.product_co de/due_date/quantit y/customer_order_no (顧客別CSVで設定され る場合あり)

2) ライン展開 (受注管理メニュー「展開実行」)

操作	API	参照 (テーブル/列)	計算/処理	保存先 (テーブル/列)
----	-----	-------------	-------	--------------

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
ライン展開	POST /line-demands/expand/	OrderLine.quantity/due_date/order.order_type RoutingStep.line/lead_time_days/duration_min/hierarchy_path/output_product CalendarDay.is_working_day	LT廻りでplan_date算出 forecast_qty/firm_qty集計 plan_qty=forecast+firm 進捗率計算	LineDemand.line_id/routing_step_id/product_id/product_code/plan_date forecast_qty/firm_qty/plan_qty/plan_progress/actual_progress/order_numbers

3) 生産計画入力 (/production/plan-input)

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
ライン選択	なし	なし	画面初期化のみ	なし
取り込み	POST /line-backlogs/pickup/ → POST /line-backlogs/recalculate_inventory/ (include_progress=false, final_only=true)	最終品: OrderLine 中間品: BOMItem 後工程 LineBacklog.plan_qty or LineGanttPlan.plan_qty RoutingStep.lead_time_days CalendarDay	order_qty/demand_qty_plan更新 → 在庫/計画在庫再計算 （ライン最終品のみ、進度は更新しない）	LineBacklog.sequence_no=0 の order_qty/demand_qty_plan stock_qty/planned_stock_qty/actual_shipment_qty

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
保存	POST /line-plans/save/ → POST /line-backlogs/expand_processes/ → POST /line-backlogs/recalculate_inventory/ (include_progress=false, final_only=true) → POST /line-gantt-plans/generate/	LinePlan入力値 Routing/BOM LineBacklog既存	計画保存 → 工程展開 → 在庫/計画在庫再計算 （ ライン最終品のみ 、進度は更新しない） → 工程ガント生成	LinePlan.plan_qty/sequence_no/plan_id LineBacklog.plan_qty ほか LineBacklog.stock_qty/planned_stock_qty/actual_shipment_qty LineGanttPlan.processes_plan

工程ガント表示	GET /line-gantt-plans/（空なら POST /line-gantt-plans/generate/）	LineGanttPlan.processes_plan/start_datetime/end_datetime	読込表示 embedded時はデータなしで自動生成	LineGanttPlan.*（自動生成時のみ）
---------	--	--	-------------------------------------	--------------------------

4) 在庫/残量一覧 (/production/inventory)

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
フィルター入力	なし	なし	ライン/工程/品番のいずれかを入力（画面フィルタ）	なし
更新	GET /line-backlogs/	LineBacklog	データ取得のみ（高速）	なし

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
再計算	POST /line-backlogs/ pickup/ → POST /line-backlog s/recalculate_scrap/ → POST /line-backlog s/recalculate_invent ory/ （表示中のラインのみ、 並列実行）	LineBacklog OrderLine BOMItem ScrapRecord/ScrapRec ordDetail	需要再計算（pickup） + 仕損再集計 + 在庫/計画在庫/進度を 再計算	LineBacklog.order_qt y/demand_qty_plan/fo recast_order_qty/fir m_order_qty/scrap_qt y LineBacklog.stock_qt y/planned_stock_qty/ progress_qty/actual_ shipment_qty

2. 在庫・計画在庫・進度の実行タイミング

- 生産計画入力「保存」：
`linePlans.save` → `expand_processes` → `recalculate_inventory (include_progress=false, final_only=true)` → `line-gantt-plans.generate`
- 生産計画入力「取り込み」：
`recalculate_inventory` は `include_progress=false, final_only=true`（ライン最終品のみ、進度は更新しない）
- 在庫/残量一覧「在庫再計算」：
 期間・ライン単位で **全製品の進捗含め再計算**（`final_only=false`）
- 在庫/残量一覧「調整数保存」：
`line-backlogs.save` 後に `recalculate_inventory` 再実行

3. 計算ロジック（要点）

実在庫（stock_qty）

- 日次計算:
 $\text{前営業日実在庫} + \text{実績生産} - \text{出庫} + \text{調整} - \text{仕損}$
- 出庫:
 最終品は確定受注（`OrderLine`）をLT遡り、非最終品は親製品の実績をBOM展開

- 起点:
前々営業日の在庫を初期値として前日以降を再計算（8:00日替わり対応）

計画在庫（planned_stock_qty）

- 過去: 実績ベース、今日以降: 計画ベース
- 日次計算:
前営業日計画在庫 + 計画 or 実績生産 - 計画出庫 + 調整 - 仕損
- 起点:
前々営業日の実在庫を初期値として使用

進度（progress_qty）

- 式:
前日進度 + 実績 - 需要(LineDemand) + 調整 - 仕損
- 需要:
確定優先、無ければ内示
- 起点:
前々営業日の進度を初期値に前日以降を再計算

5) 生産実績入力 (/production/mobile-input)

操作	API	参照（テーブル/列）	計算/処理	保存先（テーブル/列）
実績登録	POST /process-realtime-records/	Process Product Line CalendarDay（8時日替わり判定）	計画日決定（8時境目） → ProcessRealtimeRecord作成 → LineBacklog.actual_qty更新 → 出庫自動反映	ProcessRealtimeRecord.* LineBacklog.actual_qty LineBacklog.actual_shipment_qty（後工程→前工程）

操作	API	参照 (テーブル/列)	計算/処理	保存先 (テーブル/列)
仕損登録	POST /process-reality-me-records/ (event_type='SCRAP')	BOMItem (展開用) StockAllocation	ScrapRecord作成 → BOM展開して ScrapRecordDetail作成 → 在庫控除	ScrapRecord.* ScrapRecordDetail.* StockAllocation.current_stock
仕損判定	POST /process-reality-me-records/{id}/scrap-disposition/	ScrapRecord ScrapRecordDetail	RETURN: 戻し処理 (在庫 復帰) CONFIRM_SCRAP: 仕損確定	ScrapRecord.disposition_status StockAllocation.current_stock

重要ポイント:

- **8時日替わり:** 8:00未満の記録は前日扱い
- **自動連動:** 実績保存と同時にLineBacklog.actual_qty/actual_shipment_qtyが即時更新
- **在庫・計画在庫・進度の再計算は自動では実行されない:** 在庫残量画面の「在庫再計算」で明示的に実行が必要

4. 主要参照ファイル

- pm_backend/apps/orders/core/views.py
- pm_backend/apps/orders/core/services/csv_import.py
- pm_backend/apps/production/services/order_expansion.py
- pm_backend/apps/production/views.py
- pm_backend/apps/production/inventory/inventory_calculator.py
- pm_backend/apps/production/inventory/progress_calculator.py
- pm-ui/src/views/orders/CSVUpload.vue
- pm-ui/src/views/orders/OrderMenu.vue
- pm-ui/src/views/production/ProductionPlanInput.vue
- pm-ui/src/views/production/ProcessGanttView.vue
- pm-ui/src/views/inventory/ProductionInventory.vue
- 仕様書/生産計画仕様書.md
- 仕様書/在庫計算仕様書.md

- 仕様書/scheduling_logic_v3.md